

REST_API_SPECIFICATION

Helix OTA — REST API Specification

Version: 1.0.0-mvp **Date:** 2026-03-04 **Status:** Final Draft **Author:** Helix OTA Engineering Team

Table of Contents

1. [API Overview](#)
 2. [Authentication & Authorization APIs](#)
 3. [Device Management APIs](#)
 4. [Update / Artifact Management APIs](#)
 5. [Update Check API \(Device-Facing\)](#)
 6. [Rollout Management APIs](#)
 7. [Telemetry & Reporting APIs](#)
 8. [Dashboard WebSocket API](#)
 9. [Appendix A — Go Struct Definitions](#)
 10. [Appendix B — Error Code Reference](#)
 11. [Appendix C — Rate Limit Reference](#)
-

1. API Overview

1.1 Base URL

Environment	Base URL
Production	https://api.helix-ota.io
Staging	https://api.staging.helix-ota.io
Development	http://localhost:8080

All paths in this document are relative to the Base URL.

1.2 API Versioning

The API uses URL-path versioning. The current version is **v1**.

https://api.helix-ota.io/api/v1/{resource}

Version v1 is stable; breaking changes will introduce v2 while v1 is maintained for a minimum 12-month deprecation window. Non-breaking changes (new optional fields, new endpoints) may be added within v1 at any time.

1.3 Authentication Methods

Method	Description	Used By
Bearer JWT	Authorization: Bearer <access_token>	Admin console, operator tools
mTLS	Client certificate presented during TLS handshake	Devices (self-registration, update check, telemetry)
API Key	X-API-Key: <key> header	Internal service-to-service calls

JWT access tokens expire after **15 minutes**. Refresh tokens expire after **7 days** and are single-use.

1.4 Content Types

Direction	Content Type
Request (JSON)	application/json
Request (File Upload)	multipart/form-data
Response	application/json; charset=utf-8
WebSocket	application/json (frames)

1.5 Standard Error Response Format

All error responses conform to the following JSON structure:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human-readable description of the error.",
    "details": [
      {
        "field": "device_id",
        "issue": "must be a valid UUID v4"
      }
    ],
    "request_id": "req_01HXYZABCDEF",
    "timestamp": "2026-03-04T12:00:00Z"
  }
}
```

Go struct:

```

type APIError struct {
    Code      string    `json:"code"`
    Message   string    `json:"message"`
    Details   []FieldError `json:"details,omitempty"`
    RequestID string    `json:"request_id"`
    Timestamp time.Time  `json:"timestamp"`
}

type FieldError struct {
    Field string `json:"field"`
    Issue string `json:"issue"`
}

```

1.6 Pagination

List endpoints return paginated results using cursor-based pagination.

Request Parameters:

Parameter	Type	Default	Description
limit	integer	50	Number of items per page (1–500)
cursor	string	""	Opaque cursor from previous response
sort	string	"created_at:desc"	Sort field and direction

Response Envelope:

```

{
  "data": [ ... ],
  "pagination": {
    "total_count": 1234,
    "has_more": true,
    "next_cursor": "eyJpZCI6IjAxSFhZWkFCQ0RFRiJ9",
    "limit": 50
  }
}

```

Go struct:

```

type PaginatedResponse[T any] struct {
    Data      []T      `json:"data"`
    Pagination Pagination `json:"pagination"`
}

type Pagination struct {
    TotalCount int64 `json:"total_count"`
    HasMore    bool  `json:"has_more"`
    NextCursor string `json:"next_cursor"`
    Limit      int   `json:"limit"`
}

```

1.7 Common HTTP Status Codes

Code	Meaning	Usage
200	OK	Successful read / update
201	Created	Successful resource creation
204	No Content	Successful deletion
400	Bad Request	Validation / syntax error
401	Unauthorized	Missing or invalid credentials
403	Forbidden	Insufficient permissions
404	Not Found	Resource does not exist
409	Conflict	Duplicate resource / state conflict
422	Unprocessable Entity	Semantic validation failure
429	Too Many Requests	Rate limit exceeded
500	Internal Server Error	Unexpected server error
503	Service Unavailable	Maintenance / overload

2. Authentication & Authorization APIs

2.1 POST /api/v1/auth/login

Authenticate an admin or operator and obtain JWT token pair.

Authentication: None (public endpoint)

Rate Limit: 10 requests/minute per IP address

Request Headers:

Header	Required	Description
Content-Type	Yes	application/json

Request Body:

```
{
  "username": "admin@helix.io",
  "password": "S3cur3P@ssw0rd!",
  "totp_code": "123456"
}
```

Field	Type	Required	Validation	Description
username	string	Yes	3–254 chars, email format	User email address
password	string	Yes	8–128 chars	Account password

Field	Type	Required	Validation	Description
totp_code	string	Conditional	6 digits	Required if 2FA is enabled for the account

Go struct:

```
type LoginRequest struct {
    Username string `json:"username" validate:"required,email,min=3,max=254"`
    Password string `json:"password" validate:"required,min=8,max=128"`
    TOTPCode string `json:"totp_code,omitempty" validate:"omitempty,len=6,numeric"`
}
```

Success Response (200):

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIs... ",
  "refresh_token": "dGhpcyBpcyBhIHJlZnJlc2g... ",
  "token_type": "Bearer",
  "expires_in": 900,
  "user": {
    "id": "usr_01HXYZABCDEFGH",
    "username": "admin@helix.io",
    "role": "admin",
    "totp_enabled": true
  }
}
```

Go struct:

```
type LoginResponse struct {
    AccessToken string `json:"access_token"`
    RefreshToken string `json:"refresh_token"`
    TokenType string `json:"token_type"`
    ExpiresIn int `json:"expires_in"`
    User UserInfo `json:"user"`
}

type UserInfo struct {
    ID string `json:"id"`
    Username string `json:"username"`
    Role string `json:"role"`
    TOTPEnabled bool `json:"totp_enabled"`
}
```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	

Status	Code	Description
		Malformed request body or field validation failure
401	INVALID_CREDENTIALS	Username or password is incorrect
401	TOTP_REQUIRED	Account has 2FA enabled but no TOTP code was provided
401	INVALID_TOTP_CODE	The TOTP code is incorrect or expired
403	ACCOUNT_LOCKED	Account is locked due to too many failed attempts
429	RATE_LIMIT_EXCEEDED	Too many login attempts from this IP

Example — Failed Login:

```
{
  "error": {
    "code": "INVALID_CREDENTIALS",
    "message": "The username or password you entered is incorrect.",
    "details": null,
    "request_id": "req_a1b2c3d4e5",
    "timestamp": "2026-03-04T12:01:00Z"
  }
}
```

2.2 POST /api/v1/auth/refresh

Exchange a valid refresh token for a new access/refresh token pair. The old refresh token is immediately invalidated (single-use rotation).

Authentication: None (refresh token in body)

Rate Limit: 30 requests/minute per user

Request Body:

```
{
  "refresh_token": "dGhpcyBpcyBhIHJlZnJlc2g..."
}
```

Field	Type	Required	Validation	Description
refresh_token	string	Yes	Non-empty	Previously issued refresh token

Go struct:

```
type RefreshRequest struct {
    RefreshToken string `json:"refresh_token" validate:"required"`
}
```

Success Response (200):

```
{
  "access_token": "eyJhbGciOiJSUzI1NiIs...\"",
  "refresh_token": "bmV3IHJlZnJlc2ggdG9rZW4...\"",
  "token_type": "Bearer",
  "expires_in": 900
}
```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	Missing or empty refresh_token field
401	INVALID_REFRESH_TOKEN	Token is expired, revoked, or malformed
401	TOKEN_REUSE_DETECTED	A previously used refresh token was presented (potential theft)

Security Note: If TOKEN_REUSE_DETECTED is triggered, the server invalidates **all** refresh tokens for the user and sends a security alert email. The user must re-authenticate.

2.3 POST /api/v1/auth/logout

Invalidate the current session by revoking the refresh token and blacklisting the access token.

Authentication: Bearer JWT (required)

Rate Limit: 20 requests/minute per user

Request Headers:

Header	Required	Description
Authorization	Yes	Bearer <access_token>

Request Body:

```
{
  "refresh_token": "dGhpcyBpcyBhIHJlZnJlc2g..."
}
```

Field	Type	Required	Description
refresh_token	string	No	If provided, also revoke this refresh token

Success Response (200):

```
{
  "message": "Successfully logged out"
}
```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid access token

2.4 GET /api/v1/auth/me

Retrieve the currently authenticated user's profile information.

Authentication: Bearer JWT (required)

Rate Limit: 60 requests/minute per user

Request Headers:

Header	Required	Description
Authorization	Yes	Bearer <access_token>

Success Response (200):

```
{
  "id": "usr_01HXYZABCDEF",
  "username": "admin@helix.io",
  "role": "admin",
  "totp_enabled": true,
  "last_login": "2026-03-03T18:30:00Z",
  "created_at": "2025-01-15T09:00:00Z"
}
```

Go struct:

```
type UserProfile struct {
    ID          string `json:"id"`
    Username    string `json:"username"`
    Role        string `json:"role"`
    TOTPEnabled bool   `json:"totp_enabled"`
    LastLogin   time.Time `json:"last_login"`
    CreatedAt   time.Time `json:"created_at"`
}
```


Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing, expired, or invalid access token

2.5 POST /api/v1/auth/totp/setup

Initiate TOTP 2FA enrollment. Returns a provisioning URI and secret for authenticator app setup.

Authentication: Bearer JWT (required, admin or operator role)

Rate Limit: 5 requests/minute per user

Request Body: None (empty)

Success Response (200):

```
{
  "secret": "JBSWY3DPEHPK3PXP",
  "provisioning_uri": "otpauth://totp/HelixOTA:admin@helix.io?secret=JBSWY3DPEHPK3PXP&issuer=HelixOTA",
  "backup_codes": [
    "A1B2-C3D4",
    "E5F6-G7H8",
    "I9J0-K1L2",
    "M3N4-O5P6",
    "Q7R8-S9T0",
    "U1V2-W3X4"
  ]
}
```

Go struct:

```
type TOTPSetupResponse struct {
    Secret      string `json:"secret"`
    ProvisioningURI string `json:"provisioning_uri"`
    BackupCodes []string `json:"backup_codes"`
}
```

Note: TOTP remains in a “pending” state until verified via the confirm endpoint below.

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Invalid or missing token
409	TOTP_ALREADY_ENABLED	TOTP is already fully enabled for this user

2.6 POST /api/v1/auth/totp/confirm

Confirm and activate TOTP 2FA by providing a valid TOTP code generated from the provisioning secret.

Authentication: Bearer JWT (required)

Rate Limit: 10 requests/minute per user

Request Body:

```
{
  "otp_code": "123456"
}
```

Field	Type	Required	Validation
otp_code	string	Yes	6 digits

Success Response (200):

```
{
  "message": "TOTP 2FA has been enabled successfully",
  "enabled": true
}
```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	Invalid TOTP code format
401	INVALID_TOTP_CODE	Code does not match expected value
409	TOTP_SETUP_NOT_INITIATED	No pending TOTP setup found for this user

2.7 POST /api/v1/auth/totp/disable

Disable TOTP 2FA for the current user. Requires the current password and a valid TOTP code for security.

Authentication: Bearer JWT (required)

Rate Limit: 5 requests/minute per user

Request Body:

```
{
  "password": "S3cur3P@ssw0rd!",
  "otp_code": "123456"
}
```

Field	Type	Required	Validation
password	string	Yes	Current account password
totp_code	string	Yes	6 digits, current TOTP code

Success Response (200):

```
{
  "message": "TOTP 2FA has been disabled",
  "enabled": false
}
```

Error Responses:

Status	Code	Description
401	INVALID_CREDENTIALS	Password is incorrect
401	INVALID_TOTP_CODE	TOTP code is incorrect
409	TOTP_NOT_ENABLED	TOTP is not currently enabled

3. Device Management APIs

3.1 POST /api/v1/devices/register

Device self-registration endpoint. Devices present their mTLS client certificate during the TLS handshake. The server extracts the CN (Common Name) and SANs from the certificate and associates them with the device record.

Authentication: mTLS (client certificate required)

Rate Limit: 5 requests/minute per certificate CN

Request Headers:

Header	Required	Description
Content-Type	Yes	application/json

Request Body:

```
{
  "device_id": "dev_01HWIDGET001",
  "hardware_model": "HX-2000",
  "firmware_version": "1.2.3",
  "os_type": "linux-arm64",
  "serial_number": "SN-ABC12345",
  "metadata": {
    "location": "factory-floor-a",
    "deployment_zone": "us-east-1"
  }
}
```

Field	Type	Required	Validation	Description
device_id	string	Yes	1–128 chars, ^[a-zA-Z0-9_-]+\$	Unique device identifier (client-generated)
hardware_model	string	Yes	1–64 chars	Hardware model identifier
firmware_version	string	Yes	Semver format (^\\d+\\.\\d+\\.\\d+)	Currently installed firmware version
os_type	string	Yes	One of: linux-arm64, linux-amd64, rtos-armv7	Operating system type
serial_number	string	Yes	1–64 chars	Physical serial number
metadata	object	No	Max 20 keys, values max 256 chars	Arbitrary key-value metadata

Go struct:

```

type DeviceRegisterRequest struct {
    DeviceID      string          `json:"device_id" validate:"required,min=1,max=128,alphanumunderscoredash"`
    HardwareModel string          `json:"hardware_model" validate:"required,min=1,max=64"`
    FirmwareVersion string        `json:"firmware_version" validate:"required,semver"`
    OSType        string          `json:"os_type" validate:"required,oneof=linux-arm64 linux-amd64 rtos-armv7"`
    SerialNumber  string          `json:"serial_number" validate:"required,min=1,max=64"`
    Metadata      map[string]string `json:"metadata,omitempty" validate:"max=20"`
}

```

Success Response (201):

```

{
  "device_id": "dev_01HWIDGET001",
  "status": "registered",
  "hardware_model": "HX-2000",
  "firmware_version": "1.2.3",
  "os_type": "linux-arm64",

```

```

"serial_number": "SN-ABC12345",
"group_id": null,
"metadata": {
  "location": "factory-floor-a",
  "deployment_zone": "us-east-1"
},
"certificate_cn": "device-01HWIDGET001.helix-ota.io",
"last_seen": "2026-03-04T12:00:00Z",
"created_at": "2026-03-04T12:00:00Z",
"updated_at": "2026-03-04T12:00:00Z"
}

```

Go struct:

```

type Device struct {
    DeviceID      string           `json:"device_id"`
    Status        string           `json:"status"`
    HardwareModel string           `json:"hardware_model"`
    FirmwareVersion string         `json:"firmware_version"`
    OSType        string           `json:"os_type"`
    SerialNumber  string           `json:"serial_number"`
    GroupID       *string          `json:"group_id"`
    Metadata      map[string]string `json:"metadata"`
    CertificateCN  string           `json:"certificate_cn"`
    LastSeen      time.Time        `json:"last_seen"`
    CreatedAt     time.Time        `json:"created_at"`
    UpdatedAt     time.Time        `json:"updated_at"`
}

```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	Request body validation failure
401	MTLS_REQUIRED	No client certificate presented
409	DEVICE_ALREADY_EXISTS	A device with this device_id is already registered
422	CERT_MISMATCH	Certificate CN does not match the declared device_id

3.2 GET /api/v1/devices

List all registered devices with pagination and filtering.

Authentication: Bearer JWT (admin or operator role)

Rate Limit: 120 requests/minute per user

Request Headers:

Header	Required	Description
Authorization	Yes	Bearer <access_token>

Query Parameters:

Parameter	Type	Required	Description
limit	integer	No	Items per page (1–500, default 50)
cursor	string	No	Pagination cursor
sort	string	No	Sort field:direction (default created_at:desc)
status	string	No	Filter by status: registered, online, offline, decommissioned
hardware_model	string	No	Filter by hardware model
os_type	string	No	Filter by OS type
group_id	string	No	Filter by group assignment
firmware_version	string	No	Filter by installed firmware version
search	string	No	Fuzzy search across device_id, serial_number, metadata

Example Request:

```
GET /api/v1/devices?
limit=20&status=online&hardware_model=HX-2000&sort=last_seen:desc
```

Success Response (200):

```
{
  "data": [
    {
      "device_id": "dev_01HWIDGET001",
      "status": "online",
      "hardware_model": "HX-2000",
      "firmware_version": "1.3.0",
      "os_type": "linux-arm64",
      "serial_number": "SN-ABC12345",
      "group_id": "grp_factory_a",
      "last_seen": "2026-03-04T11:55:00Z",
      "created_at": "2026-01-10T08:00:00Z"
    }
  ]
}
```

```
],
  "pagination": {
    "total_count": 482,
    "has_more": true,
    "next_cursor":
      "eyJjcmVhdGVkX2F0IjoiMjAyNi0wMy0wNFQxMTowMDowMFoifQ==",
    "limit": 20
  }
}
```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	Invalid query parameter value
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Viewer role cannot access this endpoint

3.3 GET /api/v1/devices/{device_id}

Retrieve full details for a specific device.

Authentication: Bearer JWT (admin, operator, or viewer role)

Rate Limit: 200 requests/minute per user

Path Parameters:

Parameter	Type	Required	Description
device_id	string	Yes	Unique device identifier

Success Response (200):

```
{
  "device_id": "dev_01HWIDGET001",
  "status": "online",
  "hardware_model": "HX-2000",
  "firmware_version": "1.3.0",
  "os_type": "linux-arm64",
  "serial_number": "SN-ABC12345",
  "group_id": "grp_factory_a",
  "metadata": {
    "location": "factory-floor-a",
    "deployment_zone": "us-east-1"
  },
  "certificate_cn": "device-01HWIDGET001.helix-ota.io",
  "last_seen": "2026-03-04T11:55:00Z",
  "current_rollback_version": null,
  "created_at": "2026-01-10T08:00:00Z",
  "updated_at": "2026-03-04T11:55:00Z"
}
```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
404	DEVICE_NOT_FOUND	No device with the given device_id

3.4 PUT /api/v1/devices/{device_id}

Update device metadata. Only the metadata field and status (for manual override) are mutable through this endpoint.

Authentication: Bearer JWT (admin or operator role)

Rate Limit: 60 requests/minute per user

Request Body:

```
{
  "metadata": {
    "location": "factory-floor-b",
    "deployment_zone": "us-west-2",
    "notes": "Relocated on 2026-03-04"
  },
  "status": "offline"
}
```

Field	Type	Required	Validation	Description
metadata	object	No	Max 20 keys, values max 256 chars	Replace all metadata
status	string	No	One of: online, offline	Override device status (cannot set decommissioned — use DELETE)

Go struct:

```
type DeviceUpdateRequest struct {
    Metadata *map[string]string `json:"metadata,omitempty" validate:"max=20"`
    Status   *string                `json:"status,omitempty" validate:"omitempty,oneof=online offline"`
}
```

Success Response (200):

```
{
  "device_id": "dev_01HWIDGET001",
  "status": "offline",
  "metadata": {
    "location": "factory-floor-b",
```



```
    "deployment_zone": "us-west-2",
    "notes": "Relocated on 2026-03-04"
  },
  "updated_at": "2026-03-04T12:10:00Z"
}
```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	Field validation failure
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Viewer role cannot update devices
404	DEVICE_NOT_FOUND	Device does not exist

3.5 DELETE /api/v1/devices/{device_id}

Decommission a device. This is a soft delete — the device record is marked decommissioned and the certificate is revoked. The device can no longer authenticate or receive updates.

Authentication: Bearer JWT (admin role only)

Rate Limit: 30 requests/minute per user

Request Body:

```
{
  "reason": "Hardware decommissioned – end of lifecycle"
}
```

Field	Type	Required	Validation	Description
reason	string	Yes	1–512 chars	Decommission reason (audit log)

Success Response (204): No body.

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Only admin role can decommission
404	DEVICE_NOT_FOUND	Device does not exist
409	DEVICE_IN_ACTIVE_ROLLOUT	Device is currently part of an active rollout; halt the rollout first

3.6 GET /api/v1/devices/{device_id}/history

Retrieve the full update history for a specific device.

Authentication: Bearer JWT (admin, operator, or viewer role)

Rate Limit: 120 requests/minute per user

Query Parameters:

Parameter	Type	Required	Description
limit	integer	No	Items per page (1–500, default 50)
cursor	string	No	Pagination cursor
status	string	No	Filter by update status: succeeded, failed, rolled_back, in_progress

Success Response (200):

```
{
  "data": [
    {
      "history_id": "his_01HABC001",
      "device_id": "dev_01HWIDGET001",
      "rollout_id": "rol_01HROLL001",
      "from_version": "1.2.3",
      "to_version": "1.3.0",
      "artifact_id": "art_01HART001",
      "status": "succeeded",
      "started_at": "2026-02-28T10:00:00Z",
      "completed_at": "2026-02-28T10:03:42Z",
      "duration_seconds": 222
    },
    {
      "history_id": "his_01HABC002",
      "device_id": "dev_01HWIDGET001",
      "rollout_id": "rol_01HROLL002",
      "from_version": "1.3.0",
      "to_version": "1.3.1",
      "artifact_id": "art_01HART002",
      "status": "failed",
      "started_at": "2026-03-01T14:00:00Z",
      "completed_at": "2026-03-01T14:01:18Z",
      "duration_seconds": 78,
      "error_message": "Checksum verification failed: expected
        abc123, got def456"
    }
  ]
}
```

```

    ],
    "pagination": {
      "total_count": 7,
      "has_more": false,
      "next_cursor": "",
      "limit": 50
    }
  }
}

```

Go struct:

```

type DeviceHistoryEntry struct {
    HistoryID      string `json:"history_id"`
    DeviceID       string `json:"device_id"`
    RolloutID      string `json:"rollout_id"`
    FromVersion    string `json:"from_version"`
    ToVersion      string `json:"to_version"`
    ArtifactID     string `json:"artifact_id"`
    Status         string `json:"status"`
    StartedAt      time.Time `json:"started_at"`
    CompletedAt    *time.Time `json:"completed_at"`
    DurationSeconds *int `json:"duration_seconds"`
    ErrorMessage   *string `json:"error_message,omitempty"`
}

```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
404	DEVICE_NOT_FOUND	Device does not exist

3.7 POST /api/v1/devices/{device_id}/rollback

Trigger an immediate rollback on a device, reverting it to the previous firmware version.

Authentication: Bearer JWT (admin or operator role)

Rate Limit: 10 requests/minute per device

Request Body:

```

{
  "target_version": "1.2.3",
  "reason": "Post-update regression: sensor calibration drift detected"
}

```

Field	Type	Required	Validation	Description
target_version	string	No	Semver format	Specific version to roll

Field	Type	Required	Validation	Description
				back to (defaults to previous version)
reason	string	Yes	1–512 chars	Reason for rollback (audit log)

Go struct:

```
type RollbackRequest struct {
    TargetVersion string `json:"target_version,omitempty"
        validate:"omitempty,semver"`
    Reason        string `json:"reason"
        validate:"required,min=1,max=512"`
}
```

Success Response (202 Accepted):

```
{
  "rollback_id": "rbk_01HROLLBACK1",
  "device_id": "dev_01HWIDGET001",
  "from_version": "1.3.0",
  "target_version": "1.2.3",
  "status": "pending",
  "created_at": "2026-03-04T12:15:00Z"
}
```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	Field validation failure
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Viewer role cannot trigger rollback
404	DEVICE_NOT_FOUND	Device does not exist
409	NO_ROLLBACK_AVAILABLE	Device has no previous version to roll back to
409	ROLLBACK_ALREADY_IN_PROGRESS	A rollback is already pending for this device
422	TARGET_VERSION_NOT_FOUND	The specified target_version artifact does not exist

3.8 PUT /api/v1/devices/{device_id}/group

Assign or reassign a device to a device group. Groups are used for targeted rollouts.

Authentication: Bearer JWT (admin or operator role)

Rate Limit: 60 requests/minute per user

Request Body:

```
{
  "group_id": "grp_factory_b"
}
```

Field	Type	Required	Validation	Description
group_id	string	Yes	1–64 chars, ^[a-zA-Z0-9_-]+\$	Group to assign device to. Pass null to unassign.

Go struct:

```
type DeviceGroupAssignment struct {
    GroupID *string `json:"group_id"
        validate:"omitempty,min=1,max=64,alphanumunderscoredash"`
}
```

Success Response (200):

```
{
  "device_id": "dev_01HWIDGET001",
  "group_id": "grp_factory_b",
  "updated_at": "2026-03-04T12:20:00Z"
}
```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	Invalid group_id format
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Viewer role cannot modify device groups
404	DEVICE_NOT_FOUND	Device does not exist
404	GROUP_NOT_FOUND	The specified group_id does not exist

4. Update / Artifact Management APIs

4.1 POST /api/v1/artifacts/upload

Upload an OTA update artifact (firmware zip). This is a multipart form-data upload. The server validates the archive structure, runs security checks, and computes cryptographic hashes.

Authentication: Bearer JWT (admin or operator role)

Rate Limit: 10 uploads/hour per user

Request Headers:

Header	Required	Description
Content-Type	Yes	multipart/form-data

Form Fields:

Field	Type	Required	Description
file	file	Yes	OTA zip archive (max 2 GB)
version	string	Yes	Semver version string for this artifact
hardware_model	string	Yes	Target hardware model (comma-separated for multiple)
os_type	string	Yes	Target OS type
release_notes	string	No	Markdown release notes (max 10,000 chars)
metadata	string	No	JSON string of additional metadata
sha256_precomputed	string	No	Client-side SHA-256 hash for integrity verification

Go struct:

```
type ArtifactUploadRequest struct {
    File          *multipart.FileHeader `form:"file"
    validate:"required"`
    Version       string                `form:"version"
    validate:"required,semver"`
}
```

```

HardwareModel    string           `form:"hardware_model"
                validate:"required"`
OSType           string           `form:"os_type"
                validate:"required,oneof=linux-arm64 linux-amd64 rtos-
                armv7"`
ReleaseNotes     string
                `form:"release_notes,omitempty" validate:"max=10000"`
Metadata         string
                `form:"metadata,omitempty"`
SHA256Precomputed string
                `form:"sha256_precomputed,omitempty"
                validate:"omitempty,sha256"`
}

```

Success Response (201):

```

{
  "artifact_id": "art_01HART003",
  "version": "1.4.0",
  "hardware_models": ["HX-2000", "HX-3000"],
  "os_type": "linux-arm64",
  "size_bytes": 67108864,
  "sha256":
    "a3f2b8c1d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0",
  "signature_valid": true,
  "validation_status": "passed",
  "storage_path": "artifacts/HX-2000/1.4.0/artifact.zip",
  "release_notes": "## v1.4.0\n\n- Fixed sensor calibration
    drift\n- Improved boot time by 15%",
  "created_by": "usr_01HXYZABCDEF",
  "created_at": "2026-03-04T12:30:00Z"
}

```

Go struct:

```

type Artifact struct {
  ArtifactID    string    `json:"artifact_id"`
  Version       string    `json:"version"`
  HardwareModels []string  `json:"hardware_models"`
  OSType        string    `json:"os_type"`
  SizeBytes     int64     `json:"size_bytes"`
  SHA256        string    `json:"sha256"`
  SignatureValid bool       `json:"signature_valid"`
  ValidationStatus string    `json:"validation_status"`
  StoragePath   string    `json:"storage_path"`
  ReleaseNotes  string    `json:"release_notes,omitempty"`
  CreatedBy     string    `json:"created_by"`
  CreatedAt     time.Time `json:"created_at"`
}

```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	

Status	Code	Description
		Missing required fields or invalid format
400	ARTIFACT_TOO_LARGE	File exceeds 2 GB maximum
400	ARTIFACT_INVALID_ARCHIVE	Zip file is corrupt or cannot be extracted
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Viewer role cannot upload artifacts
409	ARTIFACT_VERSION_EXISTS	An artifact with this version+hardware_model+os_type already exists
422	HASH_MISMATCH	Precomputed SHA-256 does not match server-computed hash
422	ARTIFACT_SIGNATURE_INVALID	Artifact digital signature verification failed

4.2 GET /api/v1/artifacts

List all uploaded artifacts with pagination and filtering.

Authentication: Bearer JWT (admin, operator, or viewer role)

Rate Limit: 120 requests/minute per user

Query Parameters:

Parameter	Type	Required	Description
limit	integer	No	Items per page (1–500, default 50)
cursor	string	No	Pagination cursor
sort	string	No	Sort (default created_at:desc)
hardware_model	string	No	Filter by hardware model
os_type	string	No	Filter by OS type
validation_status	string	No	Filter: passed, failed, pending

Success Response (200):

```
{
  "data": [
    {
      "artifact_id": "art_01HART003",
      "version": "1.4.0",
      "hardware_models": ["HX-2000", "HX-3000"],
```



```

        "os_type": "linux-arm64",
        "size_bytes": 67108864,
        "sha256": "a3f2b8c1d4e5...",
        "validation_status": "passed",
        "created_at": "2026-03-04T12:30:00Z"
    }
],
"pagination": {
    "total_count": 23,
    "has_more": true,
    "next_cursor": "eyJpZCI6ImFydF8wMUhBUlQwMDMifQ==",
    "limit": 50
}
}

```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token

4.3 GET /api/v1/artifacts/{artifact_id}

Get full details of a specific artifact.

Authentication: Bearer JWT (admin, operator, or viewer role)

Rate Limit: 200 requests/minute per user

Path Parameters:

Parameter	Type	Required	Description
artifact_id	string	Yes	Unique artifact identifier

Success Response (200):

```

{
  "artifact_id": "art_01HART003",
  "version": "1.4.0",
  "hardware_models": ["HX-2000", "HX-3000"],
  "os_type": "linux-arm64",
  "size_bytes": 67108864,
  "sha256": "a3f2b8c1d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0",
  "signature_valid": true,
  "validation_status": "passed",
  "validation_results": {
    "archive_integrity": "passed",
    "signature_verification": "passed",
    "manifest_schema": "passed",
    "payload_scan": "clean",
    "validated_at": "2026-03-04T12:30:05Z"
  }
}

```

```

    },
    "storage_path": "artifacts/HX-2000/1.4.0/artifact.zip",
    "release_notes": "## v1.4.0\n\n- Fixed sensor calibration drift\n- Improved boot time by 15%",
    "metadata": {},
    "download_count": 142,
    "created_by": "usr_01HXYZABCDEF",
    "created_at": "2026-03-04T12:30:00Z",
    "updated_at": "2026-03-04T12:30:00Z"
  }
}

```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
404	ARTIFACT_NOT_FOUND	Artifact does not exist

4.4 DELETE /api/v1/artifacts/{artifact_id}

Delete an artifact. Only artifacts not associated with any active rollout can be deleted.

Authentication: Bearer JWT (admin role only)

Rate Limit: 30 requests/minute per user

Request Body:

```

{
  "reason": "Superseded by v1.4.1 – critical hotfix release"
}

```

Field	Type	Required	Description
reason	string	Yes	Deletion reason (1–512 chars, audit log)

Success Response (204): No body.

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Only admin role can delete artifacts
404	ARTIFACT_NOT_FOUND	Artifact does not exist
409	ARTIFACT_IN_ACTIVE_ROLLOUT	Artifact is referenced by an active or paused rollout

4.5 POST /api/v1/artifacts/{artifact_id}/validate

Re-trigger validation for an artifact. Useful after a validation pipeline update or if initial validation failed due to a transient issue.

Authentication: Bearer JWT (admin or operator role)

Rate Limit: 5 requests/minute per artifact

Request Body: None (empty)

Success Response (202 Accepted):

```
{
  "artifact_id": "art_01HART003",
  "validation_status": "pending",
  "message": "Validation has been queued and will complete
              asynchronously"
}
```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Viewer role cannot trigger validation
404	ARTIFACT_NOT_FOUND	Artifact does not exist
409	VALIDATION_ALREADY_IN_PROGRESS	Validation is already running for this artifact

4.6 GET /api/v1/artifacts/{artifact_id}/download

Download the artifact binary. Returns a streaming response with the artifact zip file.

Authentication: Bearer JWT (admin, operator) or mTLS (device)

Rate Limit: 100 requests/minute per identity

Request Headers:

Header	Required	Description
Authorization or mTLS	Yes	Authentication credential
Range	No	Byte range for resumable downloads (e.g., bytes=1048576-)

Success Response (200 or 206 for partial):

Header	Value
Content-Type	application/octet-stream
Content-Length	Size of the response body
Content-Disposition	attachment; filename="artifact_1.4.0.zip"
X-Artifact-SHA256	SHA-256 hash of the full artifact
Accept-Ranges	bytes

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid credentials
404	ARTIFACT_NOT_FOUND	Artifact does not exist
416	RANGE_NOT_SATISFIABLE	Requested byte range exceeds artifact size

5. Update Check API (Device-Facing)

5.1 POST /api/v1/updates/check

This is the primary endpoint used by devices to check whether an update is available. The server evaluates active rollouts targeting the device's hardware model, OS type, and group assignment, applying rollout percentage rules.

Authentication: mTLS (client certificate required)

Rate Limit: 60 requests/minute per device

Request Headers:

Header	Required	Description
Content-Type	Yes	application/json

Request Body:

```
{
  "device_id": "dev_01HWIDGET001",
  "current_version": "1.3.0",
  "hardware_model": "HX-2000",
  "os_type": "linux-arm64",
  "group_id": "grp_factory_a",
  "last_check": "2026-03-04T06:00:00Z"
}
```

Field	Type	Required	Validation	Description
device_id	string	Yes	1–128 chars	Device identifier (must match mTLS cert CN)
current_version	string	Yes	Semver	Currently installed firmware version
hardware_model	string	Yes	1–64 chars	Device hardware model
os_type	string	Yes	Valid OS type	Operating system type
group_id	string	No	1–64 chars	Current group assignment (if any)
last_check	string	No	ISO 8601 datetime	Timestamp of last successful check

Go struct:

```

type UpdateCheckRequest struct {
    DeviceID      string `json:"device_id"
        validate:"required,min=1,max=128"`
    CurrentVersion string `json:"current_version"
        validate:"required,semver"`
    HardwareModel string `json:"hardware_model"
        validate:"required,min=1,max=64"`
    OSType        string `json:"os_type"
        validate:"required,oneof=linux-arm64 linux-amd64 rtos-armv7"`
    GroupID       string `json:"group_id,omitempty"
        validate:"omitempty,min=1,max=64"`
    LastCheck     string `json:"last_check,omitempty"
        validate:"omitempty,datetime"`
}

```

Success Response — Update Available (200):

```

{
  "update_available": true,
  "update": {
    "version": "1.4.0",
    "artifact_id": "art_01HART003",
    "download_url": "https://cdn.helix-ota.io/artifacts/HX-2000/1.4.0/artifact.zip?token=signed_url_token",
    "size_bytes": 67108864,

```

```

    "sha256":
        "a3f2b8c1d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0",
    "signature": "MEUCIQDx1234...",
    "payload_metadata": {
        "update_type": "full",
        "install_strategy": "atomic",
        "pre_install_hook": "/opt/helix/pre-install.sh",
        "post_install_hook": "/opt/helix/post-install.sh",
        "estimated_install_time_seconds": 180,
        "reboot_required": true
    },
    "release_notes": "## v1.4.0\n\n- Fixed sensor calibration drift\n- Improved boot time by 15%",
    "rollout_id": "rol_01HROLL003",
    "deadline": "2026-03-11T00:00:00Z"
}
}

```

Go struct:

```

type UpdateCheckResponse struct {
    UpdateAvailable bool        `json:"update_available"`
    Update          *UpdateInfo `json:"update,omitempty"`
}

type UpdateInfo struct {
    Version          string        `json:"version"`
    ArtifactID       string        `json:"artifact_id"`
    DownloadURL      string        `json:"download_url"`
    SizeBytes        int64         `json:"size_bytes"`
    SHA256           string        `json:"sha256"`
    Signature        string        `json:"signature"`
    PayloadMetadata  PayloadMetadata `json:"payload_metadata"`
    ReleaseNotes     string        `json:"release_notes,omitempty"`
    RolloutID        string        `json:"rollout_id"`
    Deadline         *time.Time   `json:"deadline,omitempty"`
}

type PayloadMetadata struct {
    UpdateType      string `json:"update_type"`
    InstallStrategy string `json:"install_strategy"`
    PreInstallHook  string `json:"pre_install_hook,omitempty"`
    PostInstallHook string `json:"post_install_hook,omitempty"`
    EstimatedInstallTimeSecs int `json:"estimated_install_time_seconds"`
    RebootRequired  bool `json:"reboot_required"`
}

```

Success Response — No Update Available (200):

```
{
  "update_available": false,
  "next_check_hint_seconds": 3600
}
```

Field	Type	Description
next_check_hint_seconds	integer	Suggested interval before next check (respects server-side backoff)

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	Request body validation failure
401	MTLS_REQUIRED	No client certificate presented
403	DEVICE_DECOMMISSIONED	Device has been decommissioned
404	DEVICE_NOT_REGISTERED	Device must register before checking for updates
422	CERT_DEVICE_MISMATCH	Certificate CN does not match the device_id in the request body

6. Rollout Management APIs

6.1 POST /api/v1/rollouts

Create a new rollout campaign targeting specific devices by hardware model, OS type, and optional group.

Authentication: Bearer JWT (admin or operator role)

Rate Limit: 20 requests/minute per user

Request Body:

```
{
  "name": "HX-2000 v1.4.0 Gradual Rollout",
  "artifact_id": "art_01HART003",
  "target_selector": {
    "hardware_models": ["HX-2000"],
    "os_type": "linux-arm64",
    "group_ids": ["grp_factory_a", "grp_factory_b"],
    "exclude_device_ids": ["dev_01HWIDGET999"]
  }
}
```

```
    },
    "strategy": {
      "type": "canary",
      "initial_percentage": 5,
      "increment_percentage": 10,
      "increment_interval_minutes": 60,
      "auto_advance": true,
      "success_threshold": 0.95,
      "failure_threshold": 0.05
    },
    "scheduled_start": "2026-03-05T08:00:00Z",
    "deadline": "2026-03-12T00:00:00Z",
    "metadata": {
      "jira_ticket": "OTA-1234",
      "team": "platform-infra"
    }
  }
}
```

Field	Type	Required	Validation	Description
name	string	Yes	1–256 chars	Human-readable rollout name
artifact_id	string	Yes	Valid artifact ID	Artifact to deploy
target_selector	object	Yes	See below	Targeting criteria
target_selector.hardware_models	string[]	Yes	Min 1 item	Hardware models to target
target_selector.os_type	string	Yes	Valid OS type	OS type to target
target_selector.group_ids	string[]	No		Only include devices in these groups (empty = all)
target_selector.exclude_device_ids	string[]	No	Max 1000 items	Devices to exclude
strategy	object	Yes	See below	Rollout strategy configuration
strategy.type	string	Yes	instant, canary, gradual	Rollout strategy type
strategy.initial_percentage	number	Conditional	1–100	Required for canary and gradual
strategy.increment_percentage	number	Conditional	1–100	Required for canary and gradual

Field	Type	Required	Validation	Description
strategy.increment_interval_minutes	number	Conditional	5–10080	Minutes between increments
strategy.auto_advance	boolean	No	Default: true	Auto-advance if thresholds met
strategy.success_threshold	number	No	0.0–1.0, default 0.95	Fraction of devices that must succeed
strategy.failure_threshold	number	No	0.0–1.0, default 0.05	Fraction of failures that halts rollout
scheduled_start	string	No	ISO 8601, must be in the future	When to begin the rollout
deadline	string	No	ISO 8601, must be after scheduled_start	Mandatory update deadline
metadata	object	No	Max 20 keys	Arbitrary metadata

Go struct:

```

type RolloutCreateRequest struct {
    Name          string          `json:"name"
    validate:"required,min=1,max=256"`
    ArtifactID    string          `json:"artifact_id"
    validate:"required"`
    TargetSelector TargetSelector  `json:"target_selector"
    validate:"required"`
    Strategy      RolloutStrategy `json:"strategy"
    validate:"required"`
    ScheduledStart *time.Time      `json:"scheduled_start,omitempty"`
    Deadline      *time.Time      `json:"deadline,omitempty"`
    Metadata      map[string]string `json:"metadata,omitempty"
    validate:"max=20"`
}

type TargetSelector struct {
    HardwareModels []string `json:"hardware_models"
    validate:"required,min=1"`
    OSType         string   `json:"os_type"
    validate:"required,oneof=linux-arm64 linux-amd64 rtos-armv7"`
    GroupIDs       []string `json:"group_ids,omitempty"`
    ExcludeDeviceIDs []string `json:"exclude_device_ids,omitempty" validate:"max=1000"`
}

```

```

type RolloutStrategy struct {
    Type string `json:"type"
        validate:"required,oneof=instant canary gradual"`
    InitialPercentage *int
        `json:"initial_percentage,omitempty"
        validate:"omitempty,min=1,max=100"`
    IncrementPercentage *int
        `json:"increment_percentage,omitempty"
        validate:"omitempty,min=1,max=100"`
    IncrementIntervalMins *int
        `json:"increment_interval_minutes,omitempty"
        validate:"omitempty,min=5,max=10080"`
    AutoAdvance *bool
        `json:"auto_advance,omitempty"`
    SuccessThreshold *float64
        `json:"success_threshold,omitempty"
        validate:"omitempty,min=0,max=1"`
    FailureThreshold *float64
        `json:"failure_threshold,omitempty"
        validate:"omitempty,min=0,max=1"`
}

```

Success Response (201):

```

{
  "rollout_id": "rol_01HROLL003",
  "name": "HX-2000 v1.4.0 Gradual Rollout",
  "artifact_id": "art_01HART003",
  "status": "scheduled",
  "target_selector": {
    "hardware_models": ["HX-2000"],
    "os_type": "linux-arm64",
    "group_ids": ["grp_factory_a", "grp_factory_b"],
    "exclude_device_ids": ["dev_01HWIDGET999"],
    "estimated_device_count": 312
  },
  "strategy": {
    "type": "canary",
    "initial_percentage": 5,
    "increment_percentage": 10,
    "increment_interval_minutes": 60,
    "auto_advance": true,
    "current_percentage": 0,
    "success_threshold": 0.95,
    "failure_threshold": 0.05
  },
  "scheduled_start": "2026-03-05T08:00:00Z",
  "deadline": "2026-03-12T00:00:00Z",
  "created_by": "usr_01HXYZABCDEF",
  "created_at": "2026-03-04T12:45:00Z",
  "updated_at": "2026-03-04T12:45:00Z"
}

```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	Field validation failure
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Viewer role cannot create rollouts
404	ARTIFACT_NOT_FOUND	Referenced artifact_id does not exist
409	ARTIFACT_NOT_VALIDATED	Artifact has not passed validation
422	DEADLINE_BEFORE_START	Deadline must be after scheduled_start
422	NO_TARGET_DEVICES	Target selector matches zero devices

6.2 GET /api/v1/rollouts

List all rollouts.

Authentication: Bearer JWT (admin, operator, or viewer role)

Rate Limit: 120 requests/minute per user

Query Parameters:

Parameter	Type	Required	Description
limit	integer	No	Items per page (1–500, default 50)
cursor	string	No	Pagination cursor
sort	string	No	Default created_at:desc
status	string	No	Filter: scheduled, in_progress, paused, completed, halted
artifact_id	string	No	Filter by artifact

Success Response (200):

```
{
  "data": [
    {
      "rollout_id": "rol_01HROLL003",
      "name": "HX-2000 v1.4.0 Gradual Rollout",
      "artifact_id": "art_01HART003",
      "status": "in_progress",
      "current_percentage": 25,
      "estimated_device_count": 312,
    }
  ]
}
```

```
    "created_at": "2026-03-04T12:45:00Z"
  },
],
"pagination": {
  "total_count": 8,
  "has_more": false,
  "next_cursor": "",
  "limit": 50
}
}
```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token

6.3 GET /api/v1/rollouts/{rollout_id}

Get detailed information about a specific rollout.

Authentication: Bearer JWT (admin, operator, or viewer role)

Rate Limit: 200 requests/minute per user

Path Parameters:

Parameter	Type	Required	Description
rollout_id	string	Yes	Unique rollout identifier

Success Response (200):

```
{
  "rollout_id": "rol_01HROLL003",
  "name": "HX-2000 v1.4.0 Gradual Rollout",
  "artifact_id": "art_01HART003",
  "status": "in_progress",
  "target_selector": {
    "hardware_models": ["HX-2000"],
    "os_type": "linux-arm64",
    "group_ids": ["grp_factory_a", "grp_factory_b"],
    "exclude_device_ids": ["dev_01HWIDGET999"],
    "estimated_device_count": 312
  },
  "strategy": {
    "type": "canary",
    "initial_percentage": 5,
    "increment_percentage": 10,
    "increment_interval_minutes": 60,
    "auto_advance": true,
    "current_percentage": 25,
    "success_threshold": 0.95,
  }
}
```

```

    "failure_threshold": 0.05
  },
  "progress": {
    "total_devices": 312,
    "devices_in_scope": 78,
    "devices_completed": 72,
    "devices_succeeded": 70,
    "devices_failed": 2,
    "devices_pending": 6,
    "success_rate": 0.972,
    "failure_rate": 0.028
  },
  "scheduled_start": "2026-03-05T08:00:00Z",
  "started_at": "2026-03-05T08:00:00Z",
  "deadline": "2026-03-12T00:00:00Z",
  "next_increment_at": "2026-03-05T09:00:00Z",
  "metadata": {
    "jira_ticket": "OTA-1234",
    "team": "platform-infra"
  },
  "created_by": "usr_01HXYZABCDEF",
  "created_at": "2026-03-04T12:45:00Z",
  "updated_at": "2026-03-05T08:35:00Z"
}

```

Go struct:

```

type Rollout struct {
    RolloutID      string          `json:"rollout_id"`
    Name           string          `json:"name"`
    ArtifactID     string          `json:"artifact_id"`
    Status         string          `json:"status"`
    TargetSelector TargetSelector  `json:"target_selector"`
    Strategy       RolloutStrategyDetail `json:"strategy"`
    Progress       RolloutProgress  `json:"progress"`
    ScheduledStart *time.Time      `json:"scheduled_start,omitempty"`
    StartedAt     *time.Time      `json:"started_at,omitempty"`
    Deadline      *time.Time      `json:"deadline,omitempty"`
    NextIncrementAt *time.Time      `json:"next_increment_at,omitempty"`
    Metadata      map[string]string `json:"metadata,omitempty"`
    CreatedBy     string           `json:"created_by"`
    CreatedAt     time.Time        `json:"created_at"`
    UpdatedAt     time.Time        `json:"updated_at"`
}

```

```

type RolloutProgress struct {
    TotalDevices      int `json:"total_devices"`
    DevicesInScope    int `json:"devices_in_scope"`
    DevicesCompleted  int `json:"devices_completed"`
    DevicesSucceeded  int `json:"devices_succeeded"`
    DevicesFailed     int `json:"devices_failed"`
}

```

```
    DevicesPending    int    `json:"devices_pending"`
    SuccessRate       float64 `json:"success_rate"`
    FailureRate       float64 `json:"failure_rate"`
}
```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
404	ROLLOUT_NOT_FOUND	Rollout does not exist

6.4 PUT /api/v1/rollouts/{rollout_id}

Update a rollout's configuration. Only certain fields are mutable depending on the current rollout status.

Authentication: Bearer JWT (admin or operator role)

Rate Limit: 30 requests/minute per user

Request Body:

```
{
  "name": "HX-2000 v1.4.0 Gradual Rollout - Revised",
  "strategy": {
    "increment_percentage": 15,
    "increment_interval_minutes": 120,
    "failure_threshold": 0.03
  },
  "deadline": "2026-03-15T00:00:00Z",
  "metadata": {
    "jira_ticket": "OTA-1234",
    "team": "platform-infra",
    "revision": "2"
  }
}
```

Mutability Rules:

Rollout Status	Mutable Fields
scheduled	All fields
in_progress	strategy.increment_percentage, strategy.increment_interval_minutes, strategy.failure_threshold, deadline, metadata
paused	Same as in_progress + strategy.initial_percentage
completed	metadata only
halted	None

Success Response (200):

```
{
  "rollout_id": "rol_01HROLL003",
  "name": "HX-2000 v1.4.0 Gradual Rollout – Revised",
  "status": "in_progress",
  "strategy": {
    "type": "canary",
    "current_percentage": 25,
    "increment_percentage": 15,
    "increment_interval_minutes": 120,
    "failure_threshold": 0.03
  },
  "updated_at": "2026-03-05T09:00:00Z"
}
```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	Field validation failure
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Viewer role cannot update rollouts
404	ROLLOUT_NOT_FOUND	Rollout does not exist
409	FIELD_NOT_MUTABLE	Attempted to modify a field that is not mutable in the current status

6.5 POST /api/v1/rollouts/{rollout_id}/pause

Pause an active rollout. Devices that have already started downloading or installing will continue, but no new devices will be offered the update.

Authentication: Bearer JWT (admin or operator role)

Rate Limit: 20 requests/minute per user

Request Body:

```
{
  "reason": "Elevated failure rate observed – investigating"
}
```

Field	Type	Required	Description
reason	string	Yes	Reason for pausing (1–512 chars, audit log)

Success Response (200):

```
{
  "rollout_id": "rol_01HROLL003",
  "status": "paused",
  "previous_status": "in_progress",
  "paused_at": "2026-03-05T09:15:00Z",
  "current_percentage": 25,
  "message": "Rollout paused. Devices currently updating will
              continue. No new devices will be offered the update."
}
```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Viewer role cannot pause rollouts
404	ROLLOUT_NOT_FOUND	Rollout does not exist
409	ROLLOUT_NOT_ACTIVE	Rollout is not in in_progress status (already paused/halted/completed)

6.6 POST /api/v1/rollouts/{rollout_id}/resume

Resume a paused rollout. Updates will resume from the current percentage.

Authentication: Bearer JWT (admin or operator role)

Rate Limit: 20 requests/minute per user

Request Body:

```
{
  "reason": "Root cause identified – bad sensor batch, not
            firmware. Resuming rollout."
}
```

Field	Type	Required	Description
reason	string	Yes	Reason for resuming (audit log)

Success Response (200):

```
{
  "rollout_id": "rol_01HROLL003",
  "status": "in_progress",
  "previous_status": "paused",
  "resumed_at": "2026-03-05T10:00:00Z",
  "current_percentage": 25,
  "next_increment_at": "2026-03-05T11:00:00Z",
}
```



```
    "message": "Rollout resumed. Devices will continue to receive updates."
}
```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Viewer role cannot resume rollouts
404	ROLLOUT_NOT_FOUND	Rollout does not exist
409	ROLLOUT_NOT_PAUSED	Rollout is not in paused status

6.7 POST /api/v1/rollouts/{rollout_id}/halt

Emergency halt. Immediately stops the rollout and triggers automatic rollback for all devices that received the update in this rollout. This is the most severe action.

Authentication: Bearer JWT (admin role only)

Rate Limit: 10 requests/minute per user

Request Body:

```
{
  "reason": "Critical regression: devices becoming unresponsive after update",
  "auto_rollback": true
}
```

Field	Type	Required	Description
reason	string	Yes	Emergency reason (1–1024 chars, audit log)
auto_rollback	boolean	No	Default: true. If true, all updated devices will be instructed to roll back

Go struct:

```
type RolloutHaltRequest struct {
    Reason      string `json:"reason" validate:"required,min=1,max=1024"`
    AutoRollback *bool `json:"auto_rollback,omitempty"`
}
```

Success Response (200):

```
{
  "rollout_id": "rol_01HROLL003",
  "status": "halted",
  "previous_status": "in_progress",
  "halted_at": "2026-03-05T10:30:00Z",
  "auto_rollback": true,
  "devices_to_rollback": 72,
  "rollback_initiated": true,
  "message": "Rollout halted. Automatic rollback initiated for 72
              devices."
}
```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Only admin role can halt rollouts
404	ROLLOUT_NOT_FOUND	Rollout does not exist
409	ROLLOUT_ALREADY_HALTED	Rollout is already halted
409	ROLLOUT_COMPLETED	Cannot halt a completed rollout

6.8 GET /api/v1/rollouts/{rollout_id}/progress

Get real-time progress metrics for a rollout, including per-group breakdowns.

Authentication: Bearer JWT (admin, operator, or viewer role)

Rate Limit: 300 requests/minute per user

Query Parameters:

Parameter	Type	Required	Description
include_group_breakdown	boolean	No	Include per-group progress (default: false)

Success Response (200):

```
{
  "rollout_id": "rol_01HROLL003",
  "status": "in_progress",
  "current_percentage": 25,
  "last_updated": "2026-03-05T08:35:00Z",
  "summary": {
    "total_devices": 312,
    "devices_in_scope": 78,
    "devices_notified": 78,
    "devices_downloaded": 74,
    "devices_installing": 2,
    "devices_succeeded": 70,
  }
}
```

```

        "devices_failed": 2,
        "devices_pending": 4,
        "success_rate": 0.972,
        "failure_rate": 0.028
    },
    "timeline": [
        {
            "timestamp": "2026-03-05T08:00:00Z",
            "event": "rollout_started",
            "percentage": 5,
            "devices_in_scope": 16
        },
        {
            "timestamp": "2026-03-05T09:00:00Z",
            "event": "percentage_increased",
            "percentage": 15,
            "devices_in_scope": 47
        },
        {
            "timestamp": "2026-03-05T10:00:00Z",
            "event": "percentage_increased",
            "percentage": 25,
            "devices_in_scope": 78
        }
    ],
    "group_breakdown": [
        {
            "group_id": "grp_factory_a",
            "total_devices": 200,
            "devices_succeeded": 48,
            "devices_failed": 1,
            "success_rate": 0.979
        },
        {
            "group_id": "grp_factory_b",
            "total_devices": 112,
            "devices_succeeded": 22,
            "devices_failed": 1,
            "success_rate": 0.956
        }
    ]
}

```

Go struct:

```

type RolloutProgressDetail struct {
    RolloutID      string           `json:"rollout_id"`
    Status         string          `json:"status"`
    CurrentPercentage int             `json:"current_percentage"`
    LastUpdated    time.Time       `json:"last_updated"`
    Summary        ProgressSummary `json:"summary"`
    Timeline       []ProgressTimeline `json:"timeline"`
}

```

```

        GroupBreakdown []GroupBreakdown
        `json:"group_breakdown,omitempty"`
    }

type ProgressSummary struct {
    TotalDevices      int    `json:"total_devices"`
    DevicesInScope    int    `json:"devices_in_scope"`
    DevicesNotified    int    `json:"devices_notified"`
    DevicesDownloaded int    `json:"devices_downloaded"`
    DevicesInstalling int    `json:"devices_installing"`
    DevicesSucceeded   int    `json:"devices_succeeded"`
    DevicesFailed      int    `json:"devices_failed"`
    DevicesPending     int    `json:"devices_pending"`
    SuccessRate        float64 `json:"success_rate"`
    FailureRate        float64 `json:"failure_rate"`
}

```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
404	ROLLOUT_NOT_FOUND	Rollout does not exist

7. Telemetry & Reporting APIs

7.1 POST /api/v1/telemetry/report

Device reports its current update status. This is the primary telemetry ingestion endpoint called by devices throughout the update lifecycle.

Authentication: mTLS (client certificate required)

Rate Limit: 120 requests/minute per device

Request Body:

```

{
  "device_id": "dev_01HWIDGET001",
  "rollout_id": "rol_01HROLL003",
  "artifact_id": "art_01HART003",
  "status": "downloading",
  "progress_percentage": 67,
  "current_version": "1.3.0",
  "target_version": "1.4.0",
  "error": null,
  "diagnostics": {
    "download_speed_bps": 5242880,
    "free_disk_bytes": 536870912,
    "battery_level": null,
    "uptime_seconds": 86400
  }
}

```

```

    },
    "reported_at": "2026-03-05T08:10:30Z"
}

```

Field	Type	Required	Validation	Description
device_id	string	Yes	Must match mTLS CN	Reporting device
rollout_id	string	Yes		Associated rollout
artifact_id	string	Yes		Artifact being applied
status	string	Yes	See below	Current update status
progress_percentage	integer	No	0–100	Download/install progress
current_version	string	Yes	Semver	Version currently running
target_version	string	Yes	Semver	Version being updated to
error	object	No		Error details if status is failed
diagnostics	object	No		Device diagnostic information
reported_at	string	Yes	ISO 8601	Client-side timestamp

Valid status values:

Status	Description
notified	Device has been notified of available update
downloading	Device is downloading the artifact
downloaded	Download complete, waiting to install
verifying	Verifying artifact integrity (hash, signature)
installing	Applying the update
succeeded	Update applied successfully
failed	Update failed
rolling_back	Device is rolling back to previous version
rolled_back	Rollback completed

Go struct:

```

type TelemetryReportRequest struct {
    DeviceID      string          `json:"device_id" validate:"required"`
    RolloutID     string          `json:"rollout_id" validate:"required"`
    ArtifactID    string          `json:"artifact_id" validate:"required"`
    Status        string          `json:"status" validate:"required,oneof=notified downloading downloaded verifying installing succeeded failed rolling_back rolled_back"`
    ProgressPercentage *int          `json:"progress_percentage,omitempty" validate:"omitempty,min=0,max=100"`
    CurrentVersion string          `json:"current_version" validate:"required,semver"`
    TargetVersion string          `json:"target_version" validate:"required,semver"`
    Error          *TelemetryError `json:"error,omitempty"`
    Diagnostics    *DeviceDiagnostics `json:"diagnostics,omitempty"`
    ReportedAt     time.Time       `json:"reported_at" validate:"required"`
}

type TelemetryError struct {
    Code      string `json:"code"`
    Message   string `json:"message"`
    Retryable bool   `json:"retryable"`
    StackTrace string `json:"stack_trace,omitempty"`
}

type DeviceDiagnostics struct {
    DownloadSpeedBps *int64 `json:"download_speed_bps,omitempty"`
    FreeDiskBytes    *int64 `json:"free_disk_bytes,omitempty"`
    BatteryLevel     *int    `json:"battery_level,omitempty" validate:"omitempty,min=0,max=100"`
    UptimeSeconds    *int64 `json:"uptime_seconds,omitempty"`
}

```

Success Response (202 Accepted):

```

{
  "accepted": true,
  "device_id": "dev_01HWIDGET001",
  "server_timestamp": "2026-03-05T08:10:31Z"
}

```

Error Responses:

Status	Code	Description
400	VALIDATION_ERROR	Request body validation failure

Status	Code	Description
401	MTLS_REQUIRED	No client certificate presented
403	DEVICE_DECOMMISSIONED	Device is decommissioned
404	ROLLOUT_NOT_FOUND	Referenced rollout does not exist
404	ARTIFACT_NOT_FOUND	Referenced artifact does not exist
422	STATUS_TRANSITION_INVALID	The reported status is not a valid transition from the last known status

7.2 GET /api/v1/telemetry/overview

Get fleet-wide telemetry metrics and summary.

Authentication: Bearer JWT (admin, operator, or viewer role)

Rate Limit: 60 requests/minute per user

Query Parameters:

Parameter	Type	Required	Description
period	string	No	Time window: 1h, 6h, 24h, 7d, 30d (default 24h)
hardware_model	string	No	Filter by hardware model
os_type	string	No	Filter by OS type
group_id	string	No	Filter by group

Success Response (200):

```
{
  "period": "24h",
  "generated_at": "2026-03-05T08:30:00Z",
  "fleet_summary": {
    "total_devices": 1250,
    "online_devices": 1187,
    "offline_devices": 63,
    "decommissioned_devices": 12,
    "devices_on_latest_version": 980,
    "devices_behind": 270
  },
  "update_activity": {
    "updates_initiated_24h": 312,
    "updates_succeeded_24h": 298,

```

```

        "updates_failed_24h": 8,
        "updates_in_progress": 6,
        "rollbacks_24h": 3,
        "success_rate_24h": 0.973
    },
    "active_rollouts": 3,
    "risk_indicators": {
        "stale_devices": {
            "count": 45,
            "description": "Devices that haven't checked in for 7+ days"
        },
        "high_failure_rate_hardware": [
            {
                "hardware_model": "HX-1000",
                "failure_rate": 0.12,
                "sample_size": 25
            }
        ]
    }
}

```

Go struct:

```

type TelemetryOverview struct {
    Period          string      `json:"period"`
    GeneratedAt     time.Time   `json:"generated_at"`
    FleetSummary    FleetSummary
    UpdateActivity  UpdateActivity
    ActiveRollouts  int         `json:"active_rollouts"`
    RiskIndicators  RiskIndicators `json:"risk_indicators"`
}

```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
400	VALIDATION_ERROR	Invalid period or filter value

7.3 GET /api/v1/telemetry/devices/{device_id}

Get telemetry history for a specific device.

Authentication: Bearer JWT (admin, operator, or viewer role)

Rate Limit: 120 requests/minute per user

Query Parameters:

Parameter	Type	Required	Description
limit	integer	No	

Parameter	Type	Required	Description
			Items per page (1–500, default 50)
cursor	string	No	Pagination cursor
period	string	No	Time window: 1h, 6h, 24h, 7d (default 24h)
status	string	No	Filter by telemetry status

Success Response (200):

```
{
  "device_id": "dev_01HWIDGET001",
  "device_summary": {
    "current_version": "1.3.0",
    "last_seen": "2026-03-05T08:10:31Z",
    "total_updates": 7,
    "total_failures": 1,
    "last_update_status": "succeeded",
    "last_update_at": "2026-02-28T10:03:42Z"
  },
  "data": [
    {
      "reported_at": "2026-03-05T08:10:31Z",
      "rollout_id": "rol_01HROLL003",
      "artifact_id": "art_01HART003",
      "status": "downloading",
      "progress_percentage": 67,
      "current_version": "1.3.0",
      "target_version": "1.4.0",
      "diagnostics": {
        "download_speed_bps": 5242880,
        "free_disk_bytes": 536870912
      }
    }
  ],
  "pagination": {
    "total_count": 15,
    "has_more": true,
    "next_cursor":
      "eyJyZXBvcnRlZF9hdCI6IjIwMjYtMDVUMDY6MDA6MDBaIn0=",
    "limit": 50
  }
}
```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
404	DEVICE_NOT_FOUND	Device does not exist

7.4 GET /api/v1/telemetry/failures

Get a detailed failure analysis report across the fleet.

Authentication: Bearer JWT (admin or operator role)

Rate Limit: 30 requests/minute per user

Query Parameters:

Parameter	Type	Required	Description
period	string	No	Time window: 1h, 6h, 24h, 7d, 30d (default 7d)
hardware_model	string	No	Filter by hardware model
os_type	string	No	Filter by OS type
rollout_id	string	No	Filter by rollout
error_code	string	No	Filter by specific error code
limit	integer	No	Items per page (1–500, default 50)
cursor	string	No	Pagination cursor

Success Response (200):

```
{
  "period": "7d",
  "generated_at": "2026-03-05T08:30:00Z",
  "summary": {
    "total_failures": 42,
    "unique_error_codes": 6,
    "most_common_error": {
      "code": "CHECKSUM_MISMATCH",
      "count": 18,
      "percentage": 42.8
    },
    "affected_device_count": 38,
    "failure_rate_trend": "decreasing"
  },
  "error_breakdown": [
    {
      "error_code": "CHECKSUM_MISMATCH",
      "count": 18,
      "first_seen": "2026-02-27T14:22:00Z",
      "last_seen": "2026-03-04T09:15:00Z",
      "affected_hardware_models": ["HX-2000"],
      "sample_message": "Checksum verification failed: expected abc123, got def456"
    }
  ],
}
```

```

{
  "error_code": "DISK_FULL",
  "count": 8,
  "first_seen": "2026-03-01T06:10:00Z",
  "last_seen": "2026-03-05T03:44:00Z",
  "affected_hardware_models": ["HX-1000"],
  "sample_message": "Insufficient disk space: required 64MB,
    available 12MB"
},
{
  "error_code": "SIGNATURE_INVALID",
  "count": 6,
  "first_seen": "2026-03-02T11:00:00Z",
  "last_seen": "2026-03-03T16:30:00Z",
  "affected_hardware_models": ["HX-2000", "HX-3000"],
  "sample_message": "Artifact signature verification failed:
    RSA signature does not match"
}
],
"affected_devices": {
  "data": [
    {
      "device_id": "dev_01HWIDGET042",
      "hardware_model": "HX-2000",
      "error_code": "CHECKSUM_MISMATCH",
      "failed_at": "2026-03-04T09:15:00Z",
      "rollout_id": "rol_01HROLL003"
    }
  ],
  "pagination": {
    "total_count": 38,
    "has_more": true,
    "next_cursor":
      "eyJkZXZpY2VfaWQiOiJkZXZfMDFIV0lER0VUMDQyIn0=",
    "limit": 50
  }
}
}

```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
403	FORBIDDEN	Viewer role cannot access failure reports

7.5 GET /api/v1/telemetry/success-rates

Get success rate trend data over time, suitable for charting.

Authentication: Bearer JWT (admin, operator, or viewer role)

Rate Limit: 60 requests/minute per user

Query Parameters:

Parameter	Type	Required	Description
period	string	No	Time window: 7d, 30d, 90d (default 30d)
granularity	string	No	Data point interval: 1h, 6h, 1d (default 1d)
hardware_model	string	No	Filter by hardware model
os_type	string	No	Filter by OS type
group_id	string	No	Filter by group

Success Response (200):

```
{
  "period": "30d",
  "granularity": "1d",
  "overall_success_rate": 0.964,
  "data_points": [
    {
      "timestamp": "2026-02-03T00:00:00Z",
      "total_updates": 45,
      "succeeded": 44,
      "failed": 1,
      "success_rate": 0.978
    },
    {
      "timestamp": "2026-02-04T00:00:00Z",
      "total_updates": 52,
      "succeeded": 50,
      "failed": 2,
      "success_rate": 0.962
    }
  ],
  "by_hardware_model": [
    {
      "hardware_model": "HX-2000",
      "success_rate": 0.971,
      "sample_size": 380
    },
    {
      "hardware_model": "HX-1000",
      "success_rate": 0.940,
      "sample_size": 120
    }
  ]
}
```

Go struct:

```
type SuccessRatesResponse struct {
    Period            string           `json:"period"`
    Granularity       string           `json:"granularity"`
    OverallSuccessRate float64          `json:"overall_success_rate"`
    DataPoints        []SuccessRatePoint `json:"data_points"`
    ByHardwareModel    []HardwareModelRate `json:"by_hardware_model"`
}

type SuccessRatePoint struct {
    Timestamp    time.Time `json:"timestamp"`
    TotalUpdates int       `json:"total_updates"`
    Succeeded    int       `json:"succeeded"`
    Failed       int       `json:"failed"`
    SuccessRate  float64   `json:"success_rate"`
}
```

Error Responses:

Status	Code	Description
401	UNAUTHORIZED	Missing or invalid token
400	VALIDATION_ERROR	Invalid period or granularity value

8. Dashboard WebSocket API

8.1 WS /api/v1/ws/dashboard

Establish a WebSocket connection for real-time dashboard updates. The server pushes events as they occur in the system.

Authentication: Bearer JWT passed as query parameter: ?token=<access_token>

Rate Limit: 5 concurrent connections per user

Connection URL:

wss://api.helix-ota.io/api/v1/ws/dashboard?
token=eyJhbGciOiJIUzI1NiIs...

Connection Flow:

1. Client opens WebSocket with JWT token in query parameter
2. Server validates the token and responds with a `connection_established` message
3. Server begins pushing events
4. Client must respond to ping messages within 30 seconds or the connection is dropped
5. Client may subscribe/unsubscribe to specific event types

Connection Established Message (server → client):

```
{
  "type": "connection_established",
  "connection_id": "ws_01HCONN001",
  "server_time": "2026-03-05T08:30:00Z",
  "subscribed_events": [
    "device_registered",
    "update_started",
    "update_completed",
    "update_failed",
    "rollout_progress"
  ]
}
```

8.2 Client → Server: Subscribe

```
{
  "action": "subscribe",
  "events": ["device_registered", "rollout_progress"]
}
```

8.3 Client → Server: Unsubscribe

```
{
  "action": "unsubscribe",
  "events": ["update_started"]
}
```

8.4 Client → Server: Ping

```
{
  "action": "ping",
  "timestamp": "2026-03-05T08:30:30Z"
}
```

Server → Client: Pong

```
{
  "type": "pong",
  "server_time": "2026-03-05T08:30:30Z"
}
```

8.5 Event Types

device_registered

Fired when a new device registers or re-registers.

```
{
  "type": "device_registered",
```

```

"timestamp": "2026-03-05T08:31:00Z",
"data": {
  "device_id": "dev_01HWIDGET050",
  "hardware_model": "HX-2000",
  "firmware_version": "1.2.3",
  "os_type": "linux-arm64",
  "group_id": null
}
}

```

Go struct:

```

type DeviceRegisteredEvent struct {
    Type      string      `json:"type"`
    Timestamp time.Time   `json:"timestamp"`
    Data      DeviceRegisteredData `json:"data"`
}

type DeviceRegisteredData struct {
    DeviceID      string `json:"device_id"`
    HardwareModel string `json:"hardware_model"`
    FirmwareVersion string `json:"firmware_version"`
    OSType        string `json:"os_type"`
    GroupID       *string `json:"group_id"`
}

```

update_started

Fired when a device begins downloading or installing an update.

```

{
  "type": "update_started",
  "timestamp": "2026-03-05T08:32:00Z",
  "data": {
    "device_id": "dev_01HWIDGET001",
    "rollout_id": "rol_01HROLL003",
    "artifact_id": "art_01HART003",
    "from_version": "1.3.0",
    "to_version": "1.4.0",
    "status": "downloading"
  }
}

```

Go struct:

```

type UpdateStartedEvent struct {
    Type      string      `json:"type"`
    Timestamp time.Time   `json:"timestamp"`
    Data      UpdateStartedData `json:"data"`
}

type UpdateStartedData struct {
    DeviceID string `json:"device_id"`
}

```

```

RolloutID    string `json:"rollout_id"`
ArtifactID   string `json:"artifact_id"`
FromVersion  string `json:"from_version"`
ToVersion    string `json:"to_version"`
Status       string `json:"status"`
}

```

update_completed

Fired when a device successfully completes an update.

```

{
  "type": "update_completed",
  "timestamp": "2026-03-05T08:35:00Z",
  "data": {
    "device_id": "dev_01HWIDGET001",
    "rollout_id": "rol_01HROLL003",
    "artifact_id": "art_01HART003",
    "from_version": "1.3.0",
    "to_version": "1.4.0",
    "duration_seconds": 180
  }
}

```

Go struct:

```

type UpdateCompletedEvent struct {
    Type      string      `json:"type"`
    Timestamp time.Time   `json:"timestamp"`
    Data      UpdateCompletedData `json:"data"`
}

type UpdateCompletedData struct {
    DeviceID      string `json:"device_id"`
    RolloutID     string `json:"rollout_id"`
    ArtifactID    string `json:"artifact_id"`
    FromVersion   string `json:"from_version"`
    ToVersion     string `json:"to_version"`
    DurationSeconds int    `json:"duration_seconds"`
}

```

update_failed

Fired when a device reports a failure during the update process.

```

{
  "type": "update_failed",
  "timestamp": "2026-03-05T08:36:00Z",
  "data": {
    "device_id": "dev_01HWIDGET042",
    "rollout_id": "rol_01HROLL003",
    "artifact_id": "art_01HART003",
    "from_version": "1.3.0",

```



```

        "to_version": "1.4.0",
        "error_code": "CHECKSUM_MISMATCH",
        "error_message": "Checksum verification failed: expected
                           abc123, got def456",
        "retryable": false
    }
}

```

Go struct:

```

type UpdateFailedEvent struct {
    Type      string      `json:"type"`
    Timestamp time.Time   `json:"timestamp"`
    Data      UpdateFailedData `json:"data"`
}

```

```

type UpdateFailedData struct {
    DeviceID      string `json:"device_id"`
    RolloutID     string `json:"rollout_id"`
    ArtifactID    string `json:"artifact_id"`
    FromVersion   string `json:"from_version"`
    ToVersion     string `json:"to_version"`
    ErrorCode     string `json:"error_code"`
    ErrorMessage  string `json:"error_message"`
    Retryable     bool   `json:"retryable"`
}

```

rollout_progress

Fired when a rollout's progress changes (percentage increase, pause, resume, halt, or milestone reached).

```

{
  "type": "rollout_progress",
  "timestamp": "2026-03-05T09:00:00Z",
  "data": {
    "rollout_id": "rol_01HROLL003",
    "rollout_name": "HX-2000 v1.4.0 Gradual Rollout",
    "status": "in_progress",
    "current_percentage": 35,
    "previous_percentage": 25,
    "devices_succeeded": 98,
    "devices_failed": 3,
    "success_rate": 0.970,
    "milestone": null
  }
}

```

Go struct:

```

type RolloutProgressEvent struct {
    Type      string      `json:"type"`
    Timestamp time.Time   `json:"timestamp"`
}

```

```

    Data      RolloutProgressData `json:"data"`
}

type RolloutProgressData struct {
    RolloutID      string `json:"rollout_id"`
    RolloutName     string `json:"rollout_name"`
    Status          string `json:"status"`
    CurrentPercentage int    `json:"current_percentage"`
    PreviousPercentage int    `json:"previous_percentage"`
    DevicesSucceeded int    `json:"devices_succeeded"`
    DevicesFailed   int    `json:"devices_failed"`
    SuccessRate     float64 `json:"success_rate"`
    Milestone       *string `json:"milestone"`
}

```

8.6 WebSocket Error Handling

If the JWT token expires during an active WebSocket session, the server sends:

```

{
  "type": "error",
  "code": "TOKEN_EXPIRED",
  "message": "Authentication token has expired. Please reconnect
             with a new token."
}

```

The server will close the connection with WebSocket close code 4001 (authentication failure) after 10 seconds.

WebSocket Close Codes:

Code	Meaning
4001	Authentication failure (invalid/expired token)
4002	Rate limit exceeded (too many connections)
4003	Subscription limit exceeded
4004	Protocol violation (malformed message)
4005	Server shutting down

Appendix A — Go Struct Definitions

This appendix consolidates the core data model Go structs used across the API. These structs are defined in the `internal/api/model` package of the Helix OTA codebase.

```
package model
```

```
import "time"
```

```
// =====
```

```
// Authentication
// =====

type LoginRequest struct {
    Username string `json:"username" validate:"required,email,min=3,max=254"`
    Password string `json:"password" validate:"required,min=8,max=128"`
    TOTPCode string `json:"totp_code,omitempty" validate:"omitempty,len=6,numeric"`
}

type LoginResponse struct {
    AccessToken string `json:"access_token"`
    RefreshToken string `json:"refresh_token"`
    TokenType string `json:"token_type"`
    ExpiresIn int `json:"expires_in"`
    User UserInfo `json:"user"`
}

type RefreshRequest struct {
    RefreshToken string `json:"refresh_token" validate:"required"`
}

type RefreshResponse struct {
    AccessToken string `json:"access_token"`
    RefreshToken string `json:"refresh_token"`
    TokenType string `json:"token_type"`
    ExpiresIn int `json:"expires_in"`
}

type LogoutRequest struct {
    RefreshToken string `json:"refresh_token,omitempty"`
}

type UserInfo struct {
    ID string `json:"id"`
    Username string `json:"username"`
    Role string `json:"role"`
    TOTPEnabled bool `json:"totp_enabled"`
}

type UserProfile struct {
    ID string `json:"id"`
    Username string `json:"username"`
    Role string `json:"role"`
    TOTPEnabled bool `json:"totp_enabled"`
    LastLogin time.Time `json:"last_login"`
    CreatedAt time.Time `json:"created_at"`
}

type TOTPSetupResponse struct {
    Secret string `json:"secret"`
}
```

```

    ProvisioningURI string `json:"provisioning_uri"`
    BackupCodes    []string `json:"backup_codes"`
}

type TOTPConfirmRequest struct {
    TOTPCode string `json:"totp_code"
        validate:"required,len=6,numeric"`
}

type TOTPDisableRequest struct {
    Password string `json:"password" validate:"required"`
    TOTPCode string `json:"totp_code"
        validate:"required,len=6,numeric"`
}

// =====
// Device Management
// =====

type DeviceRegisterRequest struct {
    DeviceID      string `json:"device_id"
        validate:"required,min=1,max=128,alphanumunderscoredash"`
    HardwareModel string `json:"hardware_model"
        validate:"required,min=1,max=64"`
    FirmwareVersion string `json:"firmware_version"
        validate:"required,semver"`
    OSType        string `json:"os_type"
        validate:"required,oneof=linux-arm64 linux-amd64 rtos-armv7"`
    SerialNumber  string `json:"serial_number"
        validate:"required,min=1,max=64"`
    Metadata      map[string]string `json:"metadata,omitempty"
        validate:"max=20"`
}

type Device struct {
    DeviceID      string `json:"device_id"`
    Status        string `json:"status"`
    HardwareModel string `json:"hardware_model"`
    FirmwareVersion string `json:"firmware_version"`
    OSType        string `json:"os_type"`
    SerialNumber  string `json:"serial_number"`
    GroupID       *string `json:"group_id"`
    Metadata      map[string]string `json:"metadata"`
    CertificateCN  string `json:"certificate_cn"`
    LastSeen      time.Time `json:"last_seen"`
    CreatedAt     time.Time `json:"created_at"`
    UpdatedAt     time.Time `json:"updated_at"`
}

type DeviceUpdateRequest struct {
    Metadata *map[string]string `json:"metadata,omitempty"
        validate:"max=20"`
}

```

```

        Status    *string           `json:"status,omitempty"
                validate:"omitempty,oneof=online offline"`
    }

    type DeviceDecommissionRequest struct {
        Reason string `json:"reason"
                validate:"required,min=1,max=512"`
    }

    type DeviceGroupAssignment struct {
        GroupID *string `json:"group_id"
                validate:"omitempty,min=1,max=64,alphanumunderscoredash"`
    }

    type RollbackRequest struct {
        TargetVersion string `json:"target_version,omitempty"
                validate:"omitempty,semver"`
        Reason        string `json:"reason"
                validate:"required,min=1,max=512"`
    }

    type RollbackResponse struct {
        RollbackID    string `json:"rollback_id"`
        DeviceID       string `json:"device_id"`
        FromVersion    string `json:"from_version"`
        TargetVersion  string `json:"target_version"`
        Status         string `json:"status"`
        CreatedAt      time.Time `json:"created_at"`
    }

    type DeviceHistoryEntry struct {
        HistoryID      string `json:"history_id"`
        DeviceID       string `json:"device_id"`
        RolloutID      string `json:"rollout_id"`
        FromVersion    string `json:"from_version"`
        ToVersion      string `json:"to_version"`
        ArtifactID     string `json:"artifact_id"`
        Status         string `json:"status"`
        StartedAt      time.Time `json:"started_at"`
        CompletedAt    *time.Time `json:"completed_at"`
        DurationSeconds *int `json:"duration_seconds"`
        ErrorMessage   *string `json:"error_message,omitempty"`
    }

    // =====
    // Artifact Management
    // =====

    type ArtifactUploadRequest struct {
        File          *multipart.FileHeader `form:"file"
                validate:"required"`
        Version       string                `form:"version"
                validate:"required,semver"`
    }

```

```

HardwareModel    string
    `form:"hardware_model" validate:"required"`
OSType           string
    `form:"os_type"
    validate:"required,oneof=linux-arm64 linux-amd64 rtos-
    armv7"`
ReleaseNotes     string
    `form:"release_notes,omitempty" validate:"max=10000"`
Metadata         string
    `form:"metadata,omitempty"`
SHA256Precomputed string
    `form:"sha256_precomputed,omitempty"
    validate:"omitempty,sha256"`
}

```

```

type Artifact struct {
    ArtifactID    string `json:"artifact_id"`
    Version       string `json:"version"`
    HardwareModels []string `json:"hardware_models"`
    OSType        string `json:"os_type"`
    SizeBytes     int64  `json:"size_bytes"`
    SHA256        string `json:"sha256"`
    SignatureValid bool   `json:"signature_valid"`
    ValidationStatus string `json:"validation_status"`
    StoragePath   string `json:"storage_path"`
    ReleaseNotes  string `json:"release_notes,omitempty"`
    CreatedBy     string `json:"created_by"`
    CreatedAt     time.Time `json:"created_at"`
    UpdatedAt     time.Time `json:"updated_at"`
}

```

```

type ArtifactDeleteRequest struct {
    Reason string `json:"reason"
        validate:"required,min=1,max=512"`
}

```

```

// =====
// Update Check
// =====

```

```

type UpdateCheckRequest struct {
    DeviceID    string `json:"device_id"
        validate:"required,min=1,max=128"`
    CurrentVersion string `json:"current_version"
        validate:"required,semver"`
    HardwareModel string `json:"hardware_model"
        validate:"required,min=1,max=64"`
    OSType       string `json:"os_type"
        validate:"required,oneof=linux-arm64 linux-amd64 rtos-
        armv7"`
    GroupID      string `json:"group_id,omitempty"
        validate:"omitempty,min=1,max=64"`
    LastCheck    string `json:"last_check,omitempty"
        validate:"omitempty,datetime"`
}

```

```

type UpdateCheckResponse struct {
    UpdateAvailable bool        `json:"update_available"`
    Update           *UpdateInfo `json:"update,omitempty"`
    NextCheckHintSecs *int        `json:"next_check_hint_seconds,omitempty"`
}

type UpdateInfo struct {
    Version      string        `json:"version"`
    ArtifactID   string        `json:"artifact_id"`
    DownloadURL  string        `json:"download_url"`
    SizeBytes    int64         `json:"size_bytes"`
    SHA256       string        `json:"sha256"`
    Signature    string        `json:"signature"`
    PayloadMetadata PayloadMetadata `json:"payload_metadata"`
    ReleaseNotes string        `json:"release_notes,omitempty"`
    RolloutID    string        `json:"rollout_id"`
    Deadline     *time.Time   `json:"deadline,omitempty"`
}

type PayloadMetadata struct {
    UpdateType      string `json:"update_type"`
    InstallStrategy string `json:"install_strategy"`
    PreInstallHook  string `json:"pre_install_hook,omitempty"`
    PostInstallHook string `json:"post_install_hook,omitempty"`
    EstimatedInstallTimeSecs int    `json:"estimated_install_time_seconds"`
    RebootRequired  bool   `json:"reboot_required"`
}

// =====
// Rollout Management
// =====

type RolloutCreateRequest struct {
    Name      string        `json:"name" validate:"required,min=1,max=256"`
    ArtifactID string        `json:"artifact_id" validate:"required"`
    TargetSelector TargetSelector `json:"target_selector" validate:"required"`
    Strategy   RolloutStrategy `json:"strategy" validate:"required"`
    ScheduledStart *time.Time    `json:"scheduled_start,omitempty"`
    Deadline    *time.Time    `json:"deadline,omitempty"`
    Metadata    map[string]string `json:"metadata,omitempty" validate:"max=20"`
}

type TargetSelector struct {

```

```

HardwareModels []string `json:"hardware_models"
    validate:"required,min=1"`
OSType         string    `json:"os_type"
    validate:"required,oneof=linux-arm64 linux-amd64 rtos-
armv7"`
GroupIDs       []string  `json:"group_ids,omitempty"`
ExcludeDeviceIDs []string  `json:"exclude_device_ids,omitempty" validate:"max=1000"`
}

```

```

type RolloutStrategy struct {
    Type         string    `json:"type"
    validate:"required,oneof=instant canary gradual"`
    InitialPercentage *int     `json:"initial_percentage,omitempty"
    validate:"omitempty,min=1,max=100"`
    IncrementPercentage *int     `json:"increment_percentage,omitempty"
    validate:"omitempty,min=1,max=100"`
    IncrementIntervalMins *int     `json:"increment_interval_minutes,omitempty"
    validate:"omitempty,min=5,max=10080"`
    AutoAdvance      *bool    `json:"auto_advance,omitempty"`
    SuccessThreshold *float64 `json:"success_threshold,omitempty"
    validate:"omitempty,min=0,max=1"`
    FailureThreshold *float64 `json:"failure_threshold,omitempty"
    validate:"omitempty,min=0,max=1"`
}

```

```

type Rollout struct {
    RolloutID      string    `json:"rollout_id"`
    Name           string    `json:"name"`
    ArtifactID     string    `json:"artifact_id"`
    Status         string    `json:"status"`
    TargetSelector TargetSelector `json:"target_selector"`
    Strategy       RolloutStrategyDetail `json:"strategy"`
    Progress       RolloutProgress `json:"progress"`
    ScheduledStart *time.Time    `json:"scheduled_start,omitempty"`
    StartedAt      *time.Time    `json:"started_at,omitempty"`
    CompletedAt    *time.Time    `json:"completed_at,omitempty"`
    HaltedAt       *time.Time    `json:"halted_at,omitempty"`
    Deadline       *time.Time    `json:"deadline,omitempty"`
    NextIncrementAt *time.Time    `json:"next_increment_at,omitempty"`
    Metadata       map[string]string `json:"metadata,omitempty"`
    CreatedBy      string         `json:"created_by"`
    CreatedAt      time.Time     `json:"created_at"`
}

```



```

    UpdatedAt      time.Time      `json:"updated_at"`
}

type RolloutStrategyDetail struct {
    Type            string    `json:"type"`
    InitialPercentage int       `json:"initial_percentage"`
    IncrementPercentage int      `json:"increment_percentage"`
    IncrementIntervalMins int      `json:"increment_interval_minutes"`
    AutoAdvance     bool      `json:"auto_advance"`
    CurrentPercentage int      `json:"current_percentage"`
    SuccessThreshold float64   `json:"success_threshold"`
    FailureThreshold float64   `json:"failure_threshold"`
}

type RolloutProgress struct {
    TotalDevices    int    `json:"total_devices"`
    DevicesInScope  int    `json:"devices_in_scope"`
    DevicesCompleted int    `json:"devices_completed"`
    DevicesSucceeded int    `json:"devices_succeeded"`
    DevicesFailed   int    `json:"devices_failed"`
    DevicesPending  int    `json:"devices_pending"`
    SuccessRate     float64 `json:"success_rate"`
    FailureRate     float64 `json:"failure_rate"`
}

type RolloutHaltRequest struct {
    Reason      string `json:"reason" validate:"required,min=1,max=1024"`
    AutoRollback *bool  `json:"auto_rollback,omitempty"`
}

type RolloutPauseResumeRequest struct {
    Reason string `json:"reason" validate:"required,min=1,max=512"`
}

// =====
// Telemetry
// =====

type TelemetryReportRequest struct {
    DeviceID      string    `json:"device_id" validate:"required"`
    RolloutID     string    `json:"rollout_id" validate:"required"`
    ArtifactID    string    `json:"artifact_id" validate:"required"`
    Status        string    `json:"status" validate:"required,oneof=notified downloading downloaded verifying installing succeeded failed rolling_back rolled_back"`
}

```

```

    ProgressPercentage *int
        `json:"progress_percentage,omitempty"
        validate:"omitempty,min=0,max=100"`
    CurrentVersion      string          `json:"current_version"
        validate:"required,semver"`
    TargetVersion       string          `json:"target_version"
        validate:"required,semver"`
    Error               *TelemetryError `json:"error,omitempty"`
    Diagnostics         *DeviceDiagnostics
        `json:"diagnostics,omitempty"`
    ReportedAt          time.Time       `json:"reported_at"
        validate:"required"`
}

type TelemetryError struct {
    Code      string `json:"code"`
    Message   string `json:"message"`
    Retryable bool   `json:"retryable"`
    StackTrace string `json:"stack_trace,omitempty"`
}

type DeviceDiagnostics struct {
    DownloadSpeedBps *int64 `json:"download_speed_bps,omitempty"`
    FreeDiskBytes    *int64 `json:"free_disk_bytes,omitempty"`
    BatteryLevel     *int    `json:"battery_level,omitempty"
        validate:"omitempty,min=0,max=100"`
    UptimeSeconds    *int64 `json:"uptime_seconds,omitempty"`
}

type TelemetryReportResponse struct {
    Accepted      bool   `json:"accepted"`
    DeviceID      string `json:"device_id"`
    ServerTimestamp time.Time `json:"server_timestamp"`
}

type FleetSummary struct {
    TotalDevices      int `json:"total_devices"`
    OnlineDevices     int `json:"online_devices"`
    OfflineDevices    int `json:"offline_devices"`
    DecommissionedDevices int `json:"decommissioned_devices"`
    DevicesOnLatest   int `json:"devices_on_latest_version"`
    DevicesBehind     int `json:"devices_behind"`
}

type UpdateActivity struct {
    UpdatesInitiated24h int    `json:"updates_initiated_24h"`
    UpdatesSucceeded24h int    `json:"updates_succeeded_24h"`
    UpdatesFailed24h   int    `json:"updates_failed_24h"`
    UpdatesInProgress  int    `json:"updates_in_progress"`
    Rollbacks24h       int    `json:"rollbacks_24h"`
    SuccessRate24h     float64 `json:"success_rate_24h"`
}

// =====

```

```
// Shared / Common
// =====

type APIError struct {
    Code      string    `json:"code"`
    Message   string    `json:"message"`
    Details   []FieldError `json:"details,omitempty"`
    RequestID string    `json:"request_id"`
    Timestamp time.Time  `json:"timestamp"`
}

type FieldError struct {
    Field string `json:"field"`
    Issue string `json:"issue"`
}

type PaginatedResponse[T any] struct {
    Data      []T      `json:"data"`
    Pagination Pagination `json:"pagination"`
}

type Pagination struct {
    TotalCount int64 `json:"total_count"`
    HasMore    bool  `json:"has_more"`
    NextCursor string `json:"next_cursor"`
    Limit      int   `json:"limit"`
}

```

Appendix B — Error Code Reference

Error Code	HTTP Status	Description
VALIDATION_ERROR	400	Request body or query parameter validation failure
INVALID_CREDENTIALS	401	Incorrect username or password
TOTP_REQUIRED	401	Account has 2FA enabled but no TOTP code provided
INVALID_TOTP_CODE	401	TOTP code is incorrect or expired
ACCOUNT_LOCKED	403	Account locked due to excessive failed login attempts
UNAUTHORIZED	401	Missing, expired, or invalid authentication token
MTLS_REQUIRED	401	

Error Code	HTTP Status	Description
		Client certificate not presented during TLS handshake
INVALID_REFRESH_TOKEN	401	Refresh token is expired, revoked, or malformed
TOKEN_REUSE_DETECTED	401	A previously used refresh token was presented
FORBIDDEN	403	Authenticated but lacks required role/permission
DEVICE_NOT_FOUND	404	Device with specified ID does not exist
ARTIFACT_NOT_FOUND	404	Artifact with specified ID does not exist
ROLLOUT_NOT_FOUND	404	Rollout with specified ID does not exist
GROUP_NOT_FOUND	404	Device group with specified ID does not exist
DEVICE_ALREADY_EXISTS	409	Device with this ID is already registered
CERT_MISMATCH	422	Certificate CN does not match the declared device ID
ARTIFACT_TOO_LARGE	400	Uploaded file exceeds maximum size (2 GB)
ARTIFACT_INVALID_ARCHIVE	400	Zip archive is corrupt or cannot be extracted
ARTIFACT_VERSION_EXISTS	409	Artifact with same version+model+os already exists
HASH_MISMATCH	422	Client-provided SHA-256 does not match server-computed hash
ARTIFACT_SIGNATURE_INVALID	422	Digital signature verification failed
ARTIFACT_IN_ACTIVE_ROLLOUT	409	Cannot delete artifact referenced by active rollout
ARTIFACT_NOT_VALIDATED	409	Artifact has not passed validation checks
DEVICE_IN_ACTIVE_ROLLOUT	409	

Error Code	HTTP Status	Description
		Device is part of an active rollout
DEVICE_DECOMMISSIONED	403	Device has been decommissioned
DEVICE_NOT_REGISTERED	404	Device must register before checking for updates
CERT_DEVICE_MISMATCH	422	Certificate CN does not match device_id in request
NO_ROLLBACK_AVAILABLE	409	Device has no previous version to roll back to
ROLLBACK_ALREADY_IN_PROGRESS	409	A rollback is already pending for this device
TARGET_VERSION_NOT_FOUND	422	Specified target version artifact does not exist
ROLLOUT_NOT_ACTIVE	409	Rollout is not in a state that allows the requested action
ROLLOUT_NOT_PAUSED	409	Rollout is not in paused status
ROLLOUT_ALREADY_HALTED	409	Rollout is already halted
ROLLOUT_COMPLETED	409	Rollout has completed and cannot be modified
FIELD_NOT_MUTABLE	409	Attempted to modify a non-mutable field
DEADLINE_BEFORE_START	422	Deadline timestamp must be after scheduled start
NO_TARGET_DEVICES	422	Target selector matches zero devices
VALIDATION_ALREADY_IN_PROGRESS	409	Artifact validation is already running
TOTP_ALREADY_ENABLED	409	TOTP 2FA is already enabled
TOTP_SETUP_NOT_INITIATED	409	No pending TOTP setup exists
TOTP_NOT_ENABLED	409	TOTP is not currently enabled
STATUS_TRANSITION_INVALID	422	Reported status is not a valid transition from previous status

Error Code	HTTP Status	Description
RATE_LIMIT_EXCEEDED	429	Too many requests; retry after indicated time
RANGE_NOT_SATISFIABLE	416	Requested byte range exceeds resource size

Appendix C — Rate Limit Reference

Rate limits are enforced per-identity (user, device, or IP) depending on the endpoint. When a rate limit is exceeded, the API returns a 429 Too Many Requests response with the following headers:

Header	Description
X-RateLimit-Limit	Maximum requests allowed in the current window
X-RateLimit-Remaining	Requests remaining in the current window
X-RateLimit-Reset	Unix timestamp when the rate limit window resets
Retry-After	Seconds until the client should retry

429 Response Body:

```
{
  "error": {
    "code": "RATE_LIMIT_EXCEEDED",
    "message": "Rate limit exceeded. Please retry after 45 seconds.",
    "details": null,
    "request_id": "req_x1y2z3",
    "timestamp": "2026-03-05T08:30:00Z"
  }
}
```

Rate Limit Summary Table

Endpoint	Scope	Limit
POST /auth/login	Per IP	10 req/min
POST /auth/refresh	Per user	30 req/min
POST /auth/logout	Per user	20 req/min
GET /auth/me	Per user	60 req/min
POST /auth/totp/*	Per user	5–10 req/min
POST /devices/register	Per cert CN	5 req/min
GET /devices	Per user	120 req/min

Endpoint	Scope	Limit
GET /devices/{id}	Per user	200 req/min
PUT /devices/{id}	Per user	60 req/min
DELETE /devices/{id}	Per user	30 req/min
GET /devices/{id}/history	Per user	120 req/min
POST /devices/{id}/rollback	Per device	10 req/min
PUT /devices/{id}/group	Per user	60 req/min
POST /artifacts/upload	Per user	10 req/hour
GET /artifacts	Per user	120 req/min
GET /artifacts/{id}	Per user	200 req/min
DELETE /artifacts/{id}	Per user	30 req/min
POST /artifacts/{id}/validate	Per artifact	5 req/min
GET /artifacts/{id}/download	Per identity	100 req/min
POST /updates/check	Per device	60 req/min
POST /rollouts	Per user	20 req/min
GET /rollouts	Per user	120 req/min
GET /rollouts/{id}	Per user	200 req/min
PUT /rollouts/{id}	Per user	30 req/min
POST /rollouts/{id}/pause	Per user	20 req/min
POST /rollouts/{id}/resume	Per user	20 req/min
POST /rollouts/{id}/halt	Per user	10 req/min
GET /rollouts/{id}/progress	Per user	300 req/min
POST /telemetry/report	Per device	120 req/min
GET /telemetry/overview	Per user	60 req/min
GET /telemetry/devices/{id}	Per user	120 req/min
GET /telemetry/failures	Per user	30 req/min
GET /telemetry/success-rates	Per user	60 req/min
WS /ws/dashboard	Per user	5 concurrent

End of REST API Specification — Helix OTA v1.0.0-mvp